

Variables and Data Types

Lecture 4

Instructor:

Shovon Mandal (SM), Lecturer

Department of Computer Science and Engineering (CSE),
Northern University & Business Technology Khulna (NUBTK)

Variable

- A variable provides us with named storage that our programs can manipulate.
- **Variable Declaration and Initialization:**
- `data type variable;` or `data type variable = value;`
- Example:
- `int a, b, c;` // Declares three ints, a, b, and c.
- `int a = 10, b = 10;` // Example of initialization
- `byte B = 22;` // initializes a byte type variable B.
- `double pi = 3.14159;` // declares and assigns a value of PI.
- `char a = 'a';` // the char variable a is initialized with value 'a'

Java Variables Types

- **Local variables:** Local variables are declared in methods, constructors, or blocks.
- **Instance variables:** Instance variables are declared in a class, but outside a method, constructor or any block. Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.
- **Class/Static variables:** Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block. Static variables are stored in the static memory.

Local variables Example

```
public class Test {  
    public void pupAge() {  
        int age = 0;  
        age = age + 7;  
        System.out.println("Puppy age is : " + age);  
    }  
  
    public static void main(String args[]) {  
        Test test = new Test();  
        test.pupAge();  
    }  
}
```

Java Instance Variables Example

```
public class Employee {  
    public String name;  
    private double salary;  
    public Employee (String empName) {  
        name = empName;  
    }  
    public void setSalary(double empSal) {  
        salary = empSal;  
    }  
}
```

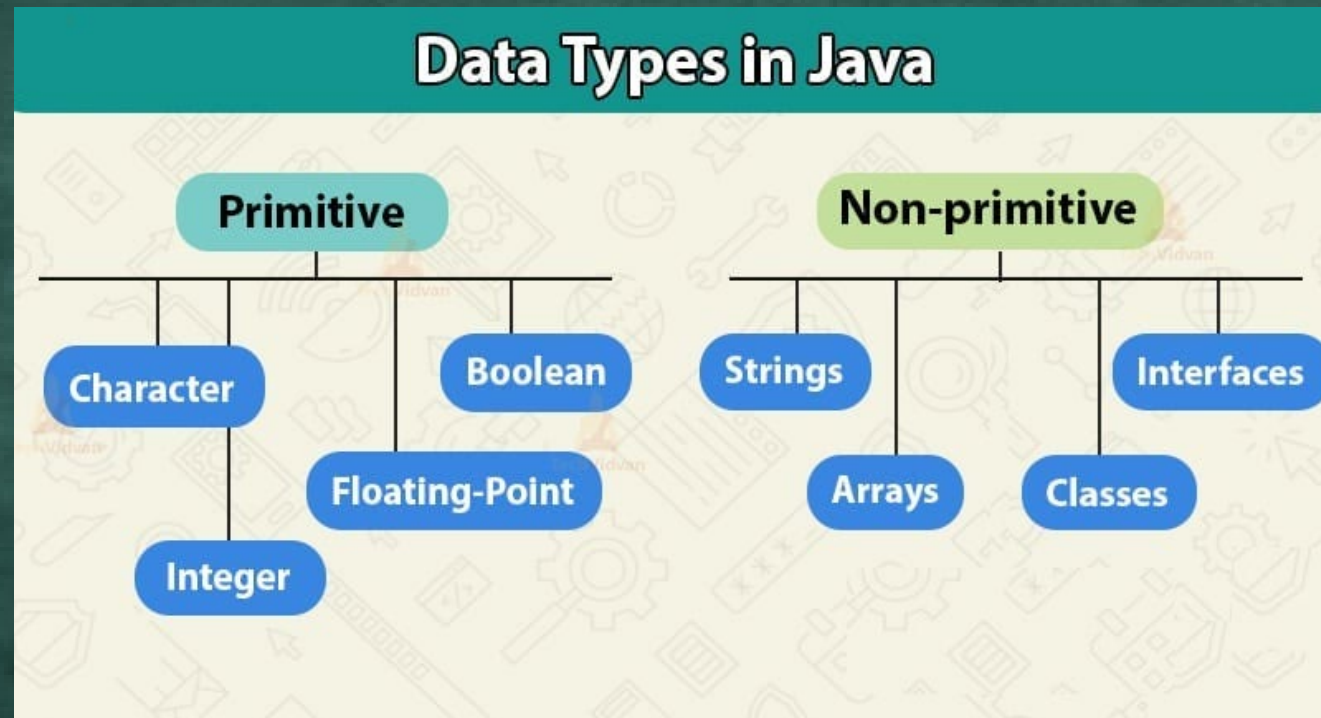
```
    public void printEmp() {  
        System.out.println("name : " + name  
);  
        System.out.println("salary : " +  
salary);  
    }  
    public static void main(String args[]) {  
        Employee empOne = new  
Employee("Ransika");  
        empOne.setSalary(1000);  
        empOne.printEmp();  
    }  
}
```

Class/Static variables Example

```
public class Employee {  
  
    // salary variable is a private static variable  
    private static double salary;  
  
    // DEPARTMENT is a constant  
    public static final String DEPARTMENT = "Development ";  
  
    public static void main(String args[]) {  
        salary = 1000;  
        System.out.println(DEPARTMENT + "average salary:" + salary);  
    }  
}
```

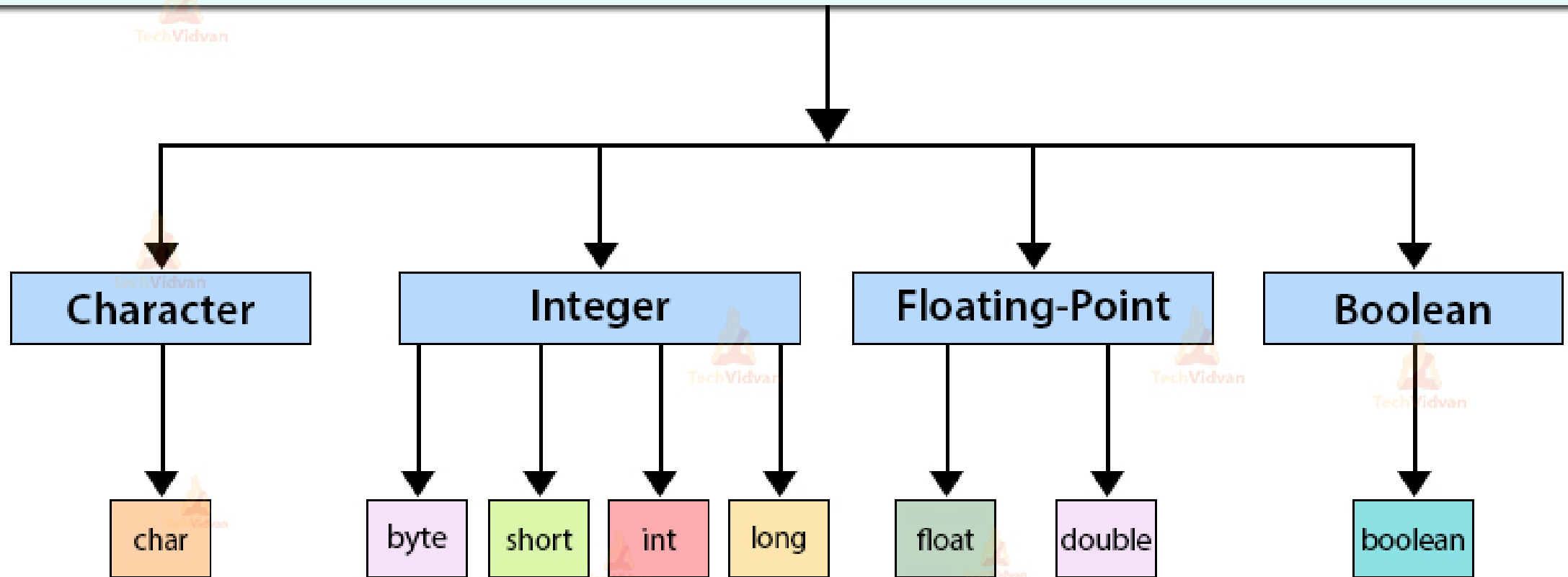

Data Types

- The Java data types are categorized into two main categories –
 - Primitive Data Types
 - Non-Primitive (Reference/Object) Data Types



Primitive Data Types in Java

Primitive Data Types in Java



Primitive Data Types in Java Cont.

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Integer Type Example

- **byte:** `byte myNum = 100;`
- **short:** `short myNum = 5000;`
- **int:** `int myNum = 100000;`
- **long:** `long myNum = 1500000000000L;`

Floating-Point Type Example

You should use a floating-point type whenever you need a number with a decimal, such as 9.99 or 3.14515.

- **float:** `float myNum = 5.75f;`
- **double:** `double myNum = 19.99d;` (not mandatory)

Note(Scientific Numbers): A floating-point number can also be a scientific number with an "e" to indicate the power of 10:

- `float f1 = 35e3f;`
- `double d1 = 12E4d;`

Use float or double?

- The precision of a floating-point value indicates how many digits the value can have after the decimal point.
- The precision of float is only six or seven decimal digits, while double variables have a precision of about 15 digits.
- Therefore, it is safer to use double for most calculations.

ASCII Table

0	NUL	16	DLE	32		48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	\$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

Characters

- In Java, the data type used to store characters is **char** that is 16-bit type.
- The standard set of characters known as ASCII still ranges from 0 to 127
- Notice that ch1 is assigned the value 88, which is the ASCII (and Unicode) value that corresponds to the letter X.

Here is a program that demonstrates **char** variables:

```
// Demonstrate char data type.
class CharDemo {
    public static void main(String args[]) {
        char ch1, ch2;

        ch1 = 88; // code for X
        ch2 = 'Y';

        System.out.print("ch1 and ch2: ");
        System.out.println(ch1 + " " + ch2);
    }
}
```

This program displays the following output:

```
ch1 and ch2: X Y
```

Boolean Types

- Very often in programming, you will need a data type that can only have one of two values, like:
- YES / NO
- ON / OFF
- TRUE / FALSE
- For this, Java has a boolean data type, which can only take the values true or false:
- **boolean one = true;**

Non-Primitive (Reference/Object) Data Types

The non-primitive data types are not predefined. The reference data types are created using defined constructors of the classes. They are used to access objects. These variables are declared to be of a specific type that cannot be changed.

Some examples of Non-Primitive Data Types:

- **String:** The string is a class in Java, and it represents the sequences of characters.
- **Arrays:** Arrays are created with the help of primitive data types and store multiple values of the same type.
- **Classes:** The classes are the user-defined data types and consist of variables and methods.
- **Interfaces:** The interfaces are abstract types that are used to specify a set of methods.

Java Type Casting (Type Conversion)

- Typecasting in Java is the process of converting one data type to another data type using the casting operator.
Syntax: <datatype> variableName = (<datatype>) value;

There are two types of Type Casting in java:

- **Widening Type Casting (Implicit)**
- **Narrow Type Casting (Explicit)**

Widening Type Casting (Implicit)

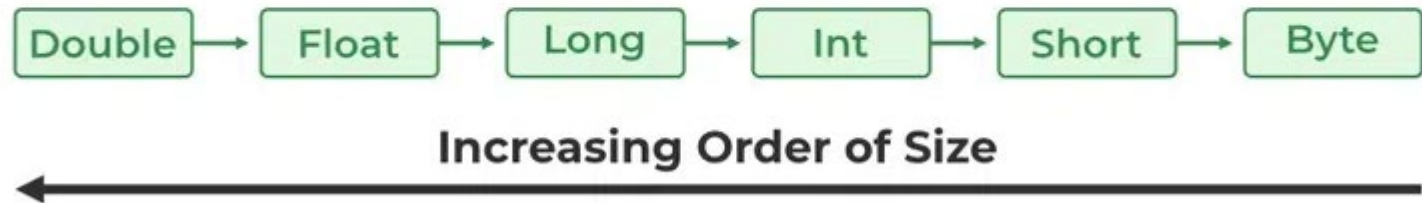
- **Widening Type Casting (Implicit):** A lower data type is transformed into a higher one by a process known as widening type casting.

Syntax: `larger_data_type variable_name = smaller_data_type_variable;`

```
int i = 10;
```

```
double d = i;
```

Widening Type Casting



Narrow Type Casting (Explicit)

- **Narrow Type Casting (Explicit):** The process of downsizing a bigger data type into a smaller one is known as narrowing type casting.

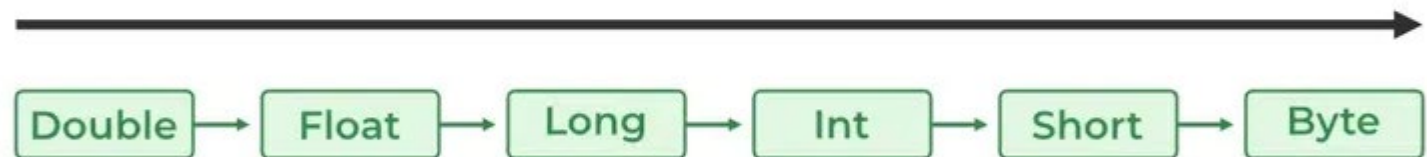
Syntax: `smaller_data_type variable_name = (smaller_data_type) larger_data_type_variable;`

```
double i = 100.245;
```

```
short j = (short)i;
```

```
int k = (int)i;
```

Explicit Type Casting Order



Thank You